

Guía de Ejercicios Resueltos: Implementando el Patrón Singleton en Java

En esta guía, exploraremos cómo aplicar el **patrón de diseño Singleton** en Java para resolver un problema común en el desarrollo de software: asegurar que una clase tenga una **única instancia** y que esta sea accesible globalmente. Nos basaremos en el caso de estudio de un sistema de pedidos online para una tienda de ropa, donde se necesita una clase `DiscountManager` (Gestor de Descuentos) para aplicar promociones de manera consistente.

Objetivo: Crear una clase `DiscountManager` que implemente el patrón Singleton, garantizando una única instancia global para gestionar todos los descuentos del sistema.

Herramientas Necesarias:

- **Java Development Kit (JDK):** Asegúrate de tenerlo instalado. Puedes descargarlo de los enlaces proporcionados en tus fuentes.
- **Apache NetBeans:** Un entorno de desarrollo integrado (IDE) para Java. Se recomienda la versión 18 por ser la más estable.

Paso 1: Comprender el Patrón Singleton y su Propósito

El patrón **Singleton** es uno de los patrones de diseño creacionales. Su propósito principal es **garantizar que una clase tenga una única instancia** y proporcionar un **punto de acceso global** a ella.

- **¿Por qué usarlo aquí?** En nuestro sistema de pedidos de ropa, el `DiscountManager` debe ser el único responsable de aplicar los descuentos. Si tuviéramos múltiples instancias, cada una podría tener un estado diferente o aplicar reglas de descuento inconsistentes, lo que llevaría a errores. El Singleton asegura que siempre trabajemos con el mismo gestor de descuentos, manteniendo la coherencia.
- **Concepto clave:** Los patrones de diseño son soluciones comprobadas a problemas habituales en el diseño de software. Conocerlos no solo te da herramientas, sino que también te ayuda a comunicarte de forma más eficiente con otros desarrolladores.

Paso 2: Configurar tu Proyecto Java en Apache NetBeans

Antes de escribir código, necesitamos un proyecto donde trabajar.

1. **Abre Apache NetBeans.**
2. Ve a `File > New Project...`
3. Selecciona `Java with Maven` o `Java with Ant` (cualquiera es válido para este ejemplo simple) y luego `Java Application`. Haz clic en `Next`.

4. Asígnale un nombre a tu proyecto, por ejemplo, `SistemaPedidosRopa` y selecciona una ubicación. Desmarca la opción `Create Main Class` por ahora, ya que la crearemos manualmente más adelante para mayor claridad.
5. Haz clic en `Finish`.

Paso 3: Crear la Clase `DiscountManager`

Esta será la clase central que gestionará los descuentos.

1. En tu proyecto `SistemaPedidosRopa` (en la vista de "Projects"), expande `Source Packages`.
2. Haz clic derecho en el paquete por defecto (o crea uno nuevo, por ejemplo, `com.tienda.descuentos`), selecciona `New > Java Class...`
3. Nombra la clase `DiscountManager` y haz clic en `Finish`.

Ahora tendrás un archivo `DiscountManager.java` vacío:

```
package com.tienda.descuentos; // O el nombre de tu paquete

public class DiscountManager {
    // Aquí implementaremos el patrón Singleton
}
```

Paso 4: Implementar un Constructor Privado

La clave del patrón Singleton es **impedir que otras partes del código creen instancias directamente** de la clase. Hacemos esto declarando el constructor como `private`.

1. Abre `DiscountManager.java`.
2. Agrega el constructor privado:

```
package com.tienda.descuentos;

public class DiscountManager {

    // Paso 4: Constructor privado
    private DiscountManager() {
        // Este constructor es privado para evitar instanciación externa.
        System.out.println("Inicializando DiscountManager (¡Solo una vez!)");
    }
}
```

- **Explicación:** Al hacer el constructor `private`, el compilador de Java no permitirá que se use el operador `new` fuera de esta clase para crear un objeto `DiscountManager`. Esto es crucial para asegurar una única instancia.

Paso 5: Declarar una Instancia Privada, Estática y Final de la Propia Clase

Necesitamos un lugar dentro de la clase `DiscountManager` para almacenar la única instancia que crearemos. Esta instancia debe ser:

- **private:** Para que solo la propia clase pueda acceder a ella directamente.
- **static:** Para que la instancia pertenezca a la clase misma, no a un objeto particular (lo que permite accederla sin crear un objeto).
- **final:** (Opcional, pero buena práctica si se inicializa de inmediato) Para asegurar que la referencia a la instancia no cambie una vez asignada.

Hay dos formas comunes de inicializarla: de forma **eager** (ansiosa, en el momento de cargar la clase) o **lazy** (perezosa, cuando se solicita por primera vez). Para este ejemplo, usaremos la inicialización perezosa, que es más común cuando el objeto puede no ser necesario de inmediato.

1. Agrega la declaración de la instancia dentro de `DiscountManager`:

```
package com.tienda.descuentos;

public class DiscountManager {

    // Paso 5: Declarar la instancia privada y estática
    private static DiscountManager instance; // Inicialmente null

    // Paso 4: Constructor privado
    private DiscountManager() {
        System.out.println("Inicializando DiscountManager (;Solo una vez!)");
    }
}
```

- **Explicación:** `instance` es una variable que contendrá nuestra única instancia de `DiscountManager`. Al ser `static`, esta variable existe una sola vez para toda la clase, sin importar cuántos intentos de acceso haya.

Paso 6: Crear el Método Estático `getInstance()`

Este método será el **punto de acceso global** a nuestra única instancia de `DiscountManager`. Será `public` y `static`.

1. Agrega el método `getInstance()` a la clase `DiscountManager`:

```
package com.tienda.descuentos;

public class DiscountManager {

    private static DiscountManager instance;

    private DiscountManager() {
        System.out.println("Inicializando DiscountManager (;Solo una vez!)");
    }
}
```

```

    }

    // Paso 6: Crear el método estático getInstance()
    public static DiscountManager getInstance() {
        // La lógica para obtener o crear la instancia va aquí.
        return instance; // Por ahora, solo devuelve la instancia (que es
null)
    }
}

```

- **Explicación:** Este método `getInstance()` es `public` para que cualquier otra clase pueda llamarlo y `static` porque lo llamamos directamente desde la clase `DiscountManager` (ej. `DiscountManager.getInstance()`) sin necesidad de crear un objeto `DiscountManager` primero.

Paso 7: Implementar la Lógica de Creación de Instancia dentro de `getInstance()` (Inicialización Perezosa)

Ahora, la lógica central del Singleton: dentro de `getInstance()`, comprobaremos si la instancia ya existe. Si no existe (es `null`), la creamos. Si ya existe, simplemente la devolvemos.

1. Modifica el método `getInstance()`:

```

package com.tienda.descuentos;

public class DiscountManager {

    private static DiscountManager instance;

    private DiscountManager() {
        System.out.println("Inicializando DiscountManager (¡Solo una
vez!)");
    }

    // Paso 7: Lógica de inicialización perezosa en getInstance()
    public static DiscountManager getInstance() {
        if (instance == null) {
            instance = new DiscountManager(); // Crea la instancia solo
si no existe
        }
        return instance;
    }

    // Métodos para gestionar descuentos (ejemplo)
    public void applyDiscount(String product, double price) {
        System.out.println("Aplicando descuento al producto: " + product
+ " con precio original " + price);
        // Aquí iría la lógica real de descuentos
        double discountedPrice = price * 0.9; // Ejemplo: 10% de
descuento
        System.out.println("Precio con descuento: " + discountedPrice);
    }
}

```

```
}
```

- **Explicación:** La condición `if (instance == null)` asegura que `new DiscountManager()` se ejecute **solo la primera vez** que se llama a `getInstance()`. Todas las llamadas subsiguientes devolverán la misma instancia existente.
- Hemos añadido un método de ejemplo `applyDiscount()` para simular la funcionalidad real del gestor.

Paso 8: Consideraciones de Concurrencia (Nota Importante)

Tus fuentes mencionan la importancia de **asegurar la atomicidad** del método `getInstance()` en entornos concurrentes. Para una aplicación de un solo hilo, el código anterior es suficiente. Sin embargo, en aplicaciones multi-hilo (donde múltiples partes del programa pueden intentar acceder al `DiscountManager` al mismo tiempo), podría haber un problema:

- Si dos hilos llaman a `getInstance()` exactamente al mismo tiempo cuando `instance` es `null`, ambos podrían pasar la condición `if (instance == null)` y crear dos instancias, rompiendo el patrón Singleton.

Para entornos concurrentes, se suelen usar mecanismos como `synchronized` o el patrón "Double-Checked Locking". Para un primer año, entender el problema es lo más importante. La solución más sencilla (aunque con un ligero impacto en rendimiento en cada llamada) es usar `synchronized` en el método `getInstance()`:

```
// Versión mejorada para entornos concurrentes (no es estrictamente
necesario para este ejercicio básico, pero importante conocerlo)
public static synchronized DiscountManager getInstance() {
    if (instance == null) {
        instance = new DiscountManager();
    }
    return instance;
}
```

- **Explicación:** La palabra clave `synchronized` garantiza que solo un hilo pueda ejecutar este método a la vez, eliminando el riesgo de crear múltiples instancias. Para este ejercicio de primer año, la versión sin `synchronized` del Paso 7 es aceptable, pero ten en cuenta esta consideración para proyectos más avanzados.

Paso 9: Demostrar el Uso del Singleton en la Clase `Main`

Ahora, vamos a probar que nuestro Singleton funciona. Crearemos una clase `Main` (o la que contenga el método `main`) para interactuar con `DiscountManager`.

1. Crea una nueva clase Java llamada `Main` (o `App`) en tu proyecto, asegurándote de marcar `public static void main(String[] args)` si tu IDE no lo hace automáticamente.

2. Agrega el siguiente código a tu método main:

```
package com.tienda.descuentos; // Asegúrate de que coincida con tu
paquete

public class Main {

    public static void main(String[] args) {
        System.out.println("--- Probando el Patrón Singleton para
DiscountManager ---");

        // Paso 9: Obtener la primera instancia de DiscountManager
        System.out.println("\nIntentando obtener la primera
instancia...");
        DiscountManager manager1 = DiscountManager.getInstance();
        System.out.println("HashCode de manager1: " +
manager1.hashCode());

        // Paso 9: Obtener la segunda instancia de DiscountManager
        System.out.println("\nIntentando obtener la segunda
instancia...");
        DiscountManager manager2 = DiscountManager.getInstance();
        System.out.println("HashCode de manager2: " +
manager2.hashCode());

        // Verificar si ambas referencias apuntan a la misma instancia
        if (manager1 == manager2) {
            System.out.println("\n¡Éxito! Ambas referencias apuntan a la
misma instancia de DiscountManager.");
        } else {
            System.out.println("\nError: Se crearon múltiples instancias
de DiscountManager.");
        }

        // Usar la instancia única para aplicar descuentos
        System.out.println("\nUsando la instancia única para aplicar
descuentos:");
        manager1.applyDiscount("Camiseta de Verano", 25.0);
        manager2.applyDiscount("Pantalones Jeans", 50.0);
    }
}
```

Paso 10: Ejecutar y Analizar el Resultado

1. Ejecuta tu clase Main (haz clic derecho en Main.java y selecciona Run File).
2. **Observa la salida en la consola:**
 - Deberías ver el mensaje **"Inicializando DiscountManager (¡Solo una vez!)"** aparecer **una única vez**, sin importar cuántas veces llares a `DiscountManager.getInstance()`. Esto confirma que solo se crea una instancia.
 - Los valores de `HashCode` para `manager1` y `manager2` deben ser **idénticos**. Esto demuestra que `manager1` y `manager2` son, de hecho, la misma instancia de objeto en la memoria.

- El mensaje "¡Éxito! Ambas referencias apuntan a la misma instancia de DiscountManager." confirmará visualmente que el patrón Singleton está funcionando correctamente.
- Verás los mensajes de aplicación de descuentos, demostrando que puedes usar la instancia única para realizar operaciones.

Resumen de los Beneficios del Patrón Singleton:

- **Control de Instancia Única:** Garantiza que solo haya una instancia de la clase, lo cual es vital para recursos compartidos como gestores de configuración, bases de datos o, en nuestro caso, gestores de descuentos.
- **Punto de Acceso Global:** Proporciona una forma conocida y centralizada para acceder a esa instancia, sin necesidad de pasar el objeto por múltiples métodos o clases.
- **Ahorro de Recursos:** Evita la creación innecesaria de múltiples objetos de una clase costosa en términos de memoria o tiempo de CPU.
- **Comunicación Mejorada:** Permite una comunicación más eficiente entre desarrolladores, ya que el término "Singleton" es un lenguaje común que describe una solución conocida.

Este ejercicio te ha permitido aplicar uno de los patrones de diseño creacionales más fundamentales en Java. ¡Sigue explorando los otros patrones como Factory Method, Abstract Factory y Builder para expandir tus herramientas de diseño de software!